

On-Chain Diploma Verification on Ethereum: Cost Trade-Offs of Per-Record Storage and Merkle-Batch Architectures

Nazar Shcherbyna

Faculty of Electrical Engineering
Warsaw University of Technology, Warsaw, Poland
nazar.shcherbyna.stud@pw.edu.pl

Bartosz Sawicki

Faculty of Electrical Engineering
Warsaw University of Technology, Warsaw, Poland

Abstract—Fake academic diplomas are a growing problem worldwide, yet checking whether a diploma is real still requires slow, manual contact with the issuing university. Blockchain registries can solve this by letting anyone verify a credential on-chain—but on Ethereum, storing data is expensive, so the way diplomas are represented in a smart contract matters greatly.

This paper examines these costs using *UniVerify*, a diploma-verification system built on Ethereum. Two designs are compared: *per-record storage*, where each diploma takes one contract slot, and *Merkle-batch issuance*, where a whole graduation class is covered by a single cryptographic root. To separate the effect of the architecture from code-level choices, four Solidity contract variants are tested with batch sizes from 1 to 4,096 diplomas.

The difference is substantial. Merkle batching cuts the per-diploma issuance cost by 98.9 % for a class of 100 students and by three orders of magnitude at 1,000 students—from \$4,820 down to \$5.09 at current Ethereum prices. Low-level EVM optimizations save another 6.6 % on verification gas, but they are far less important than the choice of architecture. A closed-form model is also derived that decomposes the optimization savings into a per-step and a constant term and matches measured gas values with errors below 0.2 %.

The main conclusion is that for universities that issue diplomas in batches, the data structure used on-chain is the most important cost factor—not low-level code tuning—and Merkle-batch registries are the practical default for blockchain-based academic credentials.

Index Terms—Ethereum, Solidity, gas optimization, Merkle tree, smart contracts, diploma verification

I. INTRODUCTION

The market for fake academic credentials is estimated at over one billion dollars per year [1], but verifying whether a diploma is genuine still depends on contacting the issuing university—a process that is slow and hard to scale across borders. Blockchain-based registries offer a different approach: once a diploma commitment is saved on-chain, anyone can check it independently, without trusting a single database.

The main obstacle is cost. On Ethereum, every permanent storage write costs about 20,000 gas. If each diploma is stored one by one, a university issuing hundreds or thousands of diplomas per graduation cycle faces very high fees. The question is therefore not *whether blockchain* can be used for

credentials, but *how diplomas should be stored on-chain so that costs stay reasonable*.

This paper addresses that question using *UniVerify*, a prototype diploma-verification system. Two approaches are compared:

- 1) **Per-record storage** — each diploma is saved in its own mapping slot and can be verified in constant time.
- 2) **Merkle batch issuance** — one Merkle root covers the whole graduation class; each diploma is verified with an off-chain proof.

The main contribution is practical: this work measures the gas cost of real, deployable Solidity contracts and provides a closed-form model that predicts optimization savings across different proof depths.

A. UniVerify System

UniVerify consists of a Solidity smart contract that handles on-chain storage and verification, and an off-chain TypeScript SDK that builds Merkle trees and generates proofs. Universities act as *issuers* who publish diploma commitments on-chain. A verifier checks a diploma by providing the document hash and, in the Merkle variant, a proof that the diploma belongs to the published batch root.

B. Contributions

- 1) A cost comparison of packed per-record storage and Merkle-batch issuance across deployment, issuance, verification, and revocation.
- 2) A four-way benchmark of Packed, Baseline, OpenZeppelin [9], and Optimized that separates architectural, library, and EVM-level effects.
- 3) A closed-form linear gas-savings model validated at six proof depths with errors below 0.2%.
- 4) An economic analysis that converts gas savings into real dollar costs for institutions.

II. RELATED WORK

Grech and Camilleri [2] identified credential verification as one of the main uses of blockchain in education. Lizcano et al. [4] proposed a decentralized credential system using IPFS

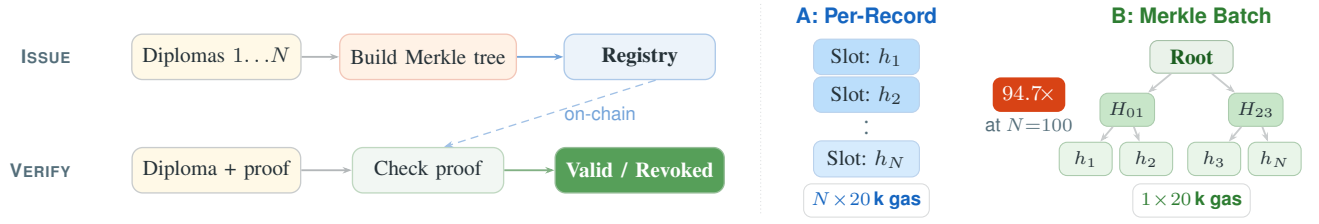


Fig. 1: UniVerify system architecture. *Left*: issuance and verification workflow (off-chain view calls cost 0 gas). *Right*: on-chain cost comparison of the two issuance models.

and Ethereum but did not analyze gas costs in detail. Alketbi et al. [3] surveyed blockchain applications in government services, including education records. The W3C Verifiable Credentials standard [10] defines a data model for tamper-proof claims but does not address cost. Arndt et al. [11] measured on-chain credential storage on Ethereum L2 networks but did not compare Merkle batching with per-record storage.

Merkle trees [5] are the standard way to prove that an element belongs to a set. OpenZeppelin’s `MerkleProof` [9] is the most widely used Solidity library for this purpose. Other approaches—sparse Merkle trees, Verkle trees [12], RSA accumulators—are not yet practical for typical EVM contracts.

Ethereum execution costs are defined by the EVM gas schedule [8], and EIP-2929 increased costs for first-time storage access [6]. Most earlier work on blockchain credentials focuses on architecture, privacy, or governance. This paper takes a different approach: it measures how specific Solidity design choices affect the gas cost of issuing and verifying diplomas in an implemented contract.

III. SYSTEM DESIGN AND METHODOLOGY

A. Two Verification Architectures

Fig. 1 shows the two approaches.

Architecture A — Per-Record Storage. Each diploma is stored in a mapping ($\text{bytes32} \Rightarrow \text{uint256}$). The issuer address (160 bits) and a revocation flag (1 bit) are packed into one 256-bit slot. Issuing a diploma requires one `SSTORE`; verifying it requires one `SLOAD` plus bit masking, which gives constant read cost. This contract is intentionally kept simple and serves as a cost baseline.

Architecture B — Merkle Batch. All diplomas in a graduation class form a Merkle tree off-chain. Only the 32-byte root is written on-chain through `issueBatchRoot`, so one storage write covers the whole batch. To verify a diploma, the caller must provide $d = \lceil \log_2 N \rceil$ sibling hashes. Each leaf uses domain separation: `keccak256(contract || chainId || issuer || batchId || docHash)`, which prevents replay across contracts and chains.

B. Contract Variants

To distinguish the effect of the architecture from that of the implementation, four contract variants were built (Table I).

TABLE I: Contract Variants and Key Design Choices

Label	Arch.	Key design choice
Packed	Per-rec.	Packed slot: issuer (160 b) + revoked (1 b) in one <code>uint256</code>
Baseline	Merkle	Hand-written proof loop; leaf via <code>abi.encodePacked</code>
OpenZeppelin	Merkle	OZ <code>MerkleProof.verifyCalldata</code>
Optimized	Merkle	Four EVM-level optimizations (T1–T4) on the hot path

C. Optimization Techniques

The **Optimized** variant targets two internal functions: `_processProof()` and `_merkleLeaf()`.

T1 — Unchecked loop. `unchecked{++i;}` removes the overflow check, saving gas on each iteration.

T2 — Scratch-space hashing. The two 32-byte inputs are placed directly in EVM scratch memory and hashed in place, avoiding the cost of `abi.encodePacked` memory allocation.

T3 — Validation splitting. The public `merkleLeaf()` validates inputs; a private `_merkleLeaf()` skips those checks and is used on internal paths.

T4 — Assembly preimage. The 112-byte leaf preimage is packed using `mstore/shl` instead of `abi.encodePacked`.

D. Experimental Setup

All measurements were performed with Foundry [7] (Solidity 0.8.20, optimizer set to 200 runs). Merkle trees are built on-chain only in the test harness; in a real deployment they would be built off-chain. Proof depths $d \in \{0, 1, 4, 8, 10, 12\}$ correspond to batch sizes from 2^0 to 2^{12} (1–4,096 diplomas). Gas values are function-level figures taken from `forge test --gas-report`.

IV. RESULTS

A. Issuance Cost

Table II compares the per-diploma issuance cost of the two architectures. Publishing a Merkle batch root costs 50,939 gas no matter how many diplomas are in the batch; the cost per diploma is therefore $50,939/N$, as shown in Fig. 2. Per-record issuance costs 48,200 gas for every single diploma.

TABLE II: Per-Diploma Issuance Cost

N	Merkle (gas)	Per-record (gas)	Ratio
1	50,939	48,200	0.95×
10	5,094	48,200	9.5×
50	1,019	48,200	47×
100	509	48,200	94.7×
1,000	51	48,200	945×

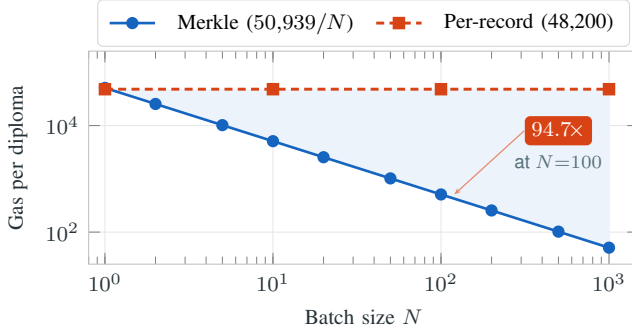


Fig. 2: Per-diploma issuance cost vs. batch size. The shaded area shows how much Merkle batching saves over per-record storage.

For a single diploma ($N=1$), batching is slightly more expensive than a direct write. Starting from $N \geq 2$, amortization becomes advantageous:

$$\text{gas}_{\text{Merkle}}(N) = \frac{50,939}{N}, \quad \text{gas}_{\text{Packed}} = 48,200 \quad (1)$$

At $N=100$, Merkle issuance is 98.9% cheaper per diploma.

B. Verification Cost

Merkle batching moves part of the cost from issuance to verification. Table III shows verification gas for depths $d \in \{0, 1, 4, 8\}$.

TABLE III: Verification Gas across Variants

Variant	Min	Avg	Max	vs. Pkd	Δ_{avg}
Packed	2,720 (constant)			—	—
Optimized	10,417	11,104	12,103	4.1×	−6.7%
OpenZeppelin	10,560	11,532	12,936	4.2×	−3.1%
Baseline	10,561	11,896	13,841	4.4×	ref.

Table III covers depths $d \in \{0, 1, 4, 8\}$; the wider range $d \in \{10, 12\}$ is shown in Table V, where the gap between **Baseline** and **Optimized** is the value of interest. Per-record verification costs only 2,720 gas—just one storage read plus bit masking. This is the central trade-off: *per-record storage is expensive to write but cheap to read; Merkle batching is cheap to write but more expensive to verify*.

Among the Merkle variants, **Optimized** has the lowest average gas, saving 6.7% versus **Baseline** and 3.7% versus **OpenZeppelin**.

C. Deployment and Revocation

Packed deploys at about one-third the gas of the Merkle variants because its logic is simpler. However, deployment happens only once, while issuance repeats every graduation cycle—so the recurring issuance cost matters much more.

TABLE IV: Deployment and State-Changing Costs (gas)

Variant	Deploy	Revoke (avg)	Issue
Packed	393,596	29,302	48,200
Optimized	1,018,273	58,303	50,939
OpenZeppelin	1,036,010	56,871	50,939
Baseline	1,142,020	59,610	51,012

D. Economic Interpretation

Using a gas price of $g=40$ gwei and $p=\$2,500/\text{ETH}$:

$$\text{USD} = \text{gas} \cdot g \cdot p \cdot 10^{-9} \quad (2)$$

This gives \$4.82 per diploma for **Packed** issuance and \$5.09 for one Merkle batch root. At graduation scale, the difference is striking: 100 diplomas cost \$482 with per-record storage versus \$5.09 with batching; 1,000 diplomas cost \$4,820 versus \$5.09. These numbers depend on network conditions: at 10 gwei / \$1,500 (low activity) the 1,000-diploma per-record cost drops to \$723, while at 80 gwei / \$4,000 (high congestion) it rises to \$15,424. The Merkle batch cost stays below \$17 in all cases, which confirms that the architectural advantage holds regardless of market conditions.

E. Closed-Form Savings Model

The gas savings between **Baseline** and **Optimized** for a single verification or revocation call can be decomposed into two terms. Let d be the proof depth (equal to the proof array length):

$$\Delta(d) = \underbrace{d \cdot 199}_{\text{T1+T2: per proof step}} + \underbrace{144}_{\text{T3+T4: constant}} \quad (3)$$

The linear term captures the per-iteration savings from T1 (unchecked loop increment) and T2 (scratch-space hashing). The constant 144 captures T3 (validation splitting, ≈ 96 gas) and T4 (assembly leaf preimage, ≈ 48 gas), both applied once per call. Table V compares the model predictions with measured gas differences from isolated `revokeFromBatch` calls, which use the same proof-verification path as `statusWithProof`; Fig. 3 visualizes the same comparison.

TABLE V: Closed-Form Model Validation

d	Leaves	Model Δ	Measured	Residual	Match
0	1	144	144	0	✓
1	2	343	343	0	✓
4	16	940	940	0	✓
8	256	1,736	1,738	2 (0.12%)	✓
10	1,024	2,134	2,137	3 (0.14%)	✓
12	4,096	2,532	2,536	4 (0.16%)	✓

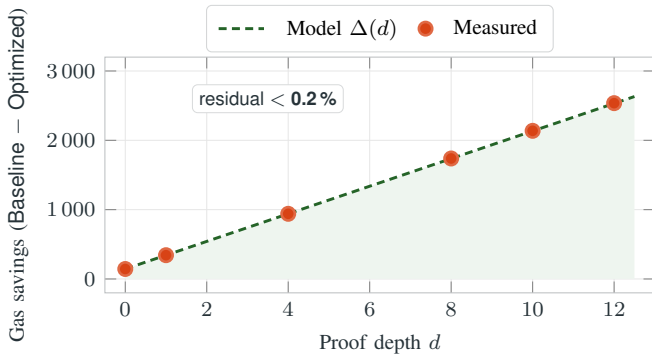


Fig. 3: Closed-form model (3) compared with measured optimization savings.

The model is linear in d , which reflects the structure of Merkle proof verification: the loop runs d times, and each remaining saving is a one-time constant. For $d \leq 4$ the model matches exactly; at higher depths a small residual appears (at most 4 gas, or 0.16%) likely due to EVM memory expansion effects.

V. DISCUSSION AND CONCLUSION

A. Engineering Trade-off

The most important design decision is the architecture. Packed has the cheapest verification but its issuance cost grows linearly with the number of diplomas. Merkle batching reverses this trade-off: issuance is nearly constant at the batch level, but each verification requires a proof. For diploma registries, this trade-off works well—issuance is a paid operation done by the institution, while verification is a read-only operation that may occur many times over the lifetime of the credential.

B. Optimization Hierarchy

Three levels of optimization can be identified:

- 1) **Architecture:** switching to Merkle batching cuts issuance cost by up to 98.9% per diploma.
- 2) **Code-level tuning:** Optimized reduces verification gas by 6.6% and deployment gas by 10.8% compared to Baseline.
- 3) **Library choice:** OpenZeppelin reduces verification gas by 3.1% compared to Baseline.

Low-level optimizations produce measurable gains but cannot make up for a costly storage model. The biggest improvement comes from removing per-record state writes.

C. Practical Implications

For small-scale or occasional issuance, per-record storage is still a good option thanks to its simple logic and constant-time lookup. For graduation-scale institutional use, Merkle batching is the better choice.

OpenZeppelin’s `MerkleProof` is a sensible default for production—it captures most of the available savings while keeping the code readable and easy to audit. The extra savings from `Optimized` are worth the added complexity only when proof verification is on a performance-critical path.

D. Limitations and Future Work

This study covers only gas costs on Ethereum L1 and does not address privacy, governance, or interoperability. All gas measurements use Foundry’s local EVM, where storage slots are warm after the first access within a transaction. On mainnet, EIP-2929 cold-access surcharges (2,100 gas per first `SLOAD`) would raise the absolute verification cost but would not change the ranking between variants. The packed per-record contract is a simplified baseline, so the comparison should be read as a cost study, not a feature-for-feature evaluation.

Future work includes running benchmarks on Layer 2 networks, studying calldata-aware designs under EIP-4844, and testing Verkle-based verification once it becomes available in production EVM.

E. Conclusion

This paper compared two deployable architectures for on-chain diploma verification and derived a closed-form model that predicts optimization savings across proof depths. The main finding is that *architecture matters more than code-level optimization*: universities that issue more than a handful of diplomas per cycle should use Merkle batching, which lowers the per-diploma cost by two orders of magnitude. For the verification path, OpenZeppelin’s `MerkleProof` gives the best balance between gas savings and code quality; hand-tuned assembly (`Optimized`) is only justified under very high throughput requirements. As credential storage moves to Layer 2 networks and EIP-4844 changes calldata pricing, the batching advantage will only grow—making Merkle-based registries the natural choice for blockchain-backed academic credentials.

REFERENCES

- [1] A. Ezell and J. Bear, “Degree Mills: The Billion-Dollar Industry That Has Sold Over a Million Fake Diplomas,” Prometheus Books, 2012.
- [2] A. Grech and A. F. Camilleri, “Blockchain in Education,” *JRC Science for Policy Report*, European Commission, 2017.
- [3] A. Alketbi, Q. Nasir, and M. A. Talib, “Blockchain for Government Services — Use Cases, Security Benefits and Challenges,” in *Proc. 15th Learn. Technol. Conf.*, 2018.
- [4] D. Lizcano, J. A. Lara, B. White, and S. Aljawarneh, “Decentralised technology for educational credentials,” *J. Comput. High. Educ.*, vol. 32, pp. 9–49, 2020.
- [5] R. C. Merkle, “A Digital Signature Based on a Conventional Encryption Function,” in *Advances in Cryptology — CRYPTO ’87*, LNCS, vol. 293, pp. 369–378, 1988.
- [6] V. Buterin et al., “EIP-2929: Gas Cost Increases for State Access Opcodes,” *Ethereum Improvement Proposals*, 2020.
- [7] Paradigm, “Foundry: Ethereum Application Development Toolkit,” <https://github.com/foundry-rs/foundry>, 2023.
- [8] G. Wood, “Ethereum: A Secure Decentralised Generalised Transaction Ledger,” *Ethereum Yellow Paper*, rev. 2024.
- [9] OpenZeppelin, “OpenZeppelin Contracts — MerkleProof,” <https://github.com/OpenZeppelin/openzeppelin-contracts>, 2026.
- [10] M. Sporny, D. Longley, and D. Chadwick, “Verifiable Credentials Data Model v2.0,” *W3C Recommendation*, 2024.
- [11] T. Arndt, F. Bünger, and U. Fiedler, “On-Chain Credential Anchoring on Ethereum Layer-2 Networks: A Cost and Latency Analysis,” in *Proc. IEEE Blockchain*, 2023, pp. 112–119.
- [12] J. Kuszmaul, “Verkle Trees,” 2018; V. Buterin et al., “EIP-6800: Ethereum state using a unified Verkle tree,” 2023.